



Web Fundamentals

WEF1VO

Teil 8: JavaScript-Grundlagen

Medientechnik und -design | WS 2025/26

1

JavaScript

Eine interpretierte Skriptsprache



Core, Client- & Server Side

Client-Side JavaScript

Zugriff auf den Browser
und Inhalte

Server-Side JavaScript

Server-Technologien
(z.B. Node.js)

Core JavaScript

Sprachfunktionen (Variablen, Datentypen etc.)

Standardisiert als ECMAScript (ECMA-262)

Version 16.0/2025 seit Juni 2025

Browser-Support: <https://compat-table.github.io/compat-table/>



**JavaScript $\neq$
Java**



Lexikalische Struktur

Die Grundregeln von JavaScript

Grundlegende Dinge



Groß- und Kleinschreibung:

`online` vs. `Online` vs. `OnLine` vs. `ONLINE`

Strichpunkt am Ende von Anweisungen:

```
a = 3;
```

```
b = 4;
```

```
// Kommentar
```

```
/* Mehrzeiliger  
   Kommentar */
```

Unicode
möglich!
Ω 😊 福

Datentypen

Zahlen, Zeichenketten und Co.

Zahlen (Number)



Ganzzahlen:

0, -3, 1234 (dezimal)

0xff, 0xa5, 0x145 (hexadezimal)

0o377, 0o27, 0o11 (oktal)

0b11111111, 0b0101, 0b010011010010 (binär)

Sehr große Ganzzahlen ($> 2^{53} - 1$):

9007199254740991n (BigInt)

Gleitkommazahlen:

3.1415

-2345.67

6.02e23 ($6,02 \cdot 10^{23}$)

Not a Number:

NaN

Zeichenketten (String)



`"random"`

String in doppelten
Anführungszeichen

`'testing'`

String in einfachen
Anführungszeichen

`""`

Leerer String

`"3.14"`

Zahl 3.14 als String

`'<p title="test">'`

Doppelte Anführungszeichen
innerhalb von einfachen

`"Hier wird\numgebrochen"`

Zeilenumbruch innerhalb der
Zeichenkette

`"<p title=\"test\">"`

Escaping bei doppelten
Anführungszeichen



Template Literale

Strings zusammenhängen (Konkatenation):

```
let name = "Welt";  
let a = 5, b = 10;  
let msgAlt = "Hallo " + name + ", Ergebnis: " + (a + b);
```

Template-Literale:

```
let msgNeu = `Hallo ${name}, Ergebnis: ${a + b}`;
```

```
let html = `

<h1>Titel</h1>  
  <p>Ein Absatz über mehrere Zeilen.</p>  
</div>`;


```



Boolean & triviale Datentypen

Boolesche Werte

`true, false`

Keinen Wert zuweisen

`null`

Kein Wert definiert

`undefined`



Arrays

Definition:

```
["Yara", "Tariq", "Alex"]  
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]  
["WEF1VO", 2025, ["YouTube", "Microsoft Teams"]] // data
```

Zugriff:

```
data[0] // WEF1VO  
data[2][1] // Microsoft Teams
```

Länge:

```
data.length // 3
```

Variablen

Werte speichern



Benennung

`[_ $A-Z]` `[_ $A-Z0-9]`

Erstes Zeichen

Jedes weitere Zeichen

A-Z und 0-9: Unicode!

```
let summe = 0;  
let numberOfPersons = 84;  
let $$bling$$ = 1000000;  
let ๖_๖ = "Oh hai!";  
let ๖_๖๖๖๖ = 42;
```

Deklaration



Deklaration:

```
let i;  
let summe, nachricht;
```

Deklaration und Zuweisung:

```
let i = 0;  
let summe = 3 + 5;  
let nachricht = "Hallo Welt!";
```

Implizite Deklaration:

```
i = 3;
```

Variablen **oldschool**



Deklaration:

```
var i;  
var summe, nachricht;  
var nachricht = "Hallo Welt!";
```

Anderer Scope – Rest gleich (Details in 27 Slides).

Dennoch: Wann immer möglich **let** (oder besser: **const**) verwenden.

Loses

Typkonzept

```
let x = 42;
```

```
...
```

```
x = "fourtytwo";
```



Konstanten



Deklaration:

```
const LIMIT = 3600;  
const DEFAULT_NAME = "John Doe";  
const ALTER = 7 + 12;
```

Nur triviale Datentypen sind immutable:

```
const myArray = [1, 2, 3];  
console.log("Vorher:", myArray); // 1,2,3  
myArray.push(4);  
console.log("Nachher:", myArray); // 1,2,3,4
```

Wert in die
Browser-Konsole
der DevTools
schreiben



*Es ist guter Stil, **wann immer möglich**,
const (anstatt **let**) zur
Variablendeklaration zu verwenden, um
zu verdeutlichen, dass **keine neue
Zuweisung** im Code vorgenommen
werden wird.*

*Nur wenn tatsächlich neue Zuweisungen
nötig sind (etwa bei Schleifen), ist **let**
vorzuziehen.*



Strikter Modus

"use strict"

Nicht mehr erlaubt (uvm):

```
i = 3;  
const oct = 0234;
```

Stattdessen:

```
const i = 3;  
const oct = 0o234;
```

Operatoren

Mit Werten arbeiten

Zuweisung



Zuweisung:

```
i = 0;
```

```
i = j = k = 0;
```

Zuweisung mit Operation:

```
i += 5; (entspricht i = i + 5)
```

```
i -= 5; (entspricht i = i - 5)
```

```
i *= 5; (entspricht i = i * 5)
```

```
i /= 5; (entspricht i = i / 5)
```

```
i %= 5; (entspricht i = i % 5)
```



Vergleich

Vergleich auf Wert:

```
3 != 4 // true
```

```
1 == "1" // true
```

Vergleich auf Wert und Typ:

```
3 !== 4 // true
```

```
1 === "1" // false
```

Vergleich auf Verhältnis:

```
3 < 4 // true
```

```
"hallo" >= "welt" // false
```

Arithmetik



Arithmetische Operatoren:

`1 + 2` // 3

`1 - 2` // -1

`3 * 4` // 12

`5 / 2` // 2.5

`5 % 2` // 1

`3 ** 2` // 9

`i++, ++i` // i + 1

`i--, --i` // i - 1

Logik



Logisches UND:

```
true && false // false
```

Logisches ODER:

```
(3 > 4) || (5 > 3) // true
```

Logisches NICHT:

```
!(3 == 3) // false
```

Konditionaler Operator



Konditionaler Operator:

```
x > 0 ? x * y : -x * y;
```

Entspricht:

```
if (x > 0) {  
    x * y;  
} else {  
    -x * y;  
}
```

Kontrollstrukturen

Abläufe im Programm steuern

Verzweigungen: **if**



```
if (name !== "") {  
    username = name;  
} else {  
    username = "JD";  
}
```

```
if (n === 1) {  
    // Code 1...  
} else if (n === 2) {  
    // Code 2...  
} else if (n === 3) {  
    // Code 3...  
} else {  
    // Restlicher Code  
}
```

Verzweigungen: **switch**



```
switch(n) {  
    case 1:  
        // Code 1...  
        break;  
    case 2:  
        // Code 2...  
        break;  
    default:  
        // Restlicher Code, wenn nichts zutrifft.  
        break;  
}
```



Schleifen

```
let count = 0;  
while (count < 10) {  
    // Mach was hier ...  
    count++;  
}
```

```
for (let count = 0;  
     count < 10;  
     count++) {  
    // Mach etwas hier ...  
}
```



for...of : Arrays iterieren

```
let namen = ["Yara", "Tariq", "Alex"];
```

```
for (const element of namen) {  
  console.log(element);  
}
```

Output in der Konsole:

Yara

Tariq

Alex

Funktionen

Wiederverwendbare Code-Blöcke



Definieren

Funktionsdeklaration

Funktionsname

```
function sayHello() {  
    console.log("Hallo Welt!");  
}
```

Nachricht in die Browser-Konsole (Dev-Tools)



Parameter

Funktionsname

Parameter

```
function saySomething(message) {  
    console.log(message);  
}
```

The diagram shows two orange arrows pointing to parts of the function syntax. One arrow points from the label 'Funktionsname' to the word 'saySomething'. The other arrow points from the label 'Parameter' to the word 'message' inside the parentheses.



Call by Value

```
const text = "Der Text bleibt."
```

```
function saySomethingWithChange(message) {  
  message += " So schauts aus."  
  console.log(message);  
}
```

← Der Text bleibt. So schauts aus.

```
saySomethingWithChange(text);  
console.log(text);
```

← Der Text bleibt.



Rückgabewerte

```
function quadrat(x) {  
    return x * x;  
}
```

Das Ergebnis kann einer Variable zugewiesen werden.

```
const result = quadrat(5); // 25
```

Funktionen aufrufen



Mit Wert als Argument:

```
quadrat(5);
```

Mit Variable als Argument:

```
saySomethingWithChange(message);
```

Ohne Argument:

```
irgendwas();
```

Mit zwei Argumenten und Zuweisung:

```
const total = addValues(5, 2);
```

Parameter

Angabe bei der Definition
der Funktion.

-VS-

Argument

Übergebener Wert beim
Aufruf der Funktion.

Aufrufen: **args** < **params**



Definition einer Funktion mit zwei Parametern:

```
function logData(first, last) {  
  console.log("Argumente:", arguments.length);  
  console.log("Vorname:", first);  
  console.log("Nachname:", last);  
}
```

Aufruf mit zwei Argumenten:

```
logData("John", "Doe");
```

Argumente: 2
Vorname: John
Nachname: Doe

Aufruf mit einem Argument:

```
logData("John");
```

Argumente: 1
Vorname: John
Nachname: undefined

Aufrufen: **args** > **params**



Definition einer Funktion mit zwei Parametern:

```
function logData(first, last) {  
  console.log("Argumente: " + arguments.length);  
  console.log("Vorname: " + first);  
  console.log("Nachname: " + last);  
  if (arguments.length > 2) {  
    for (let i = 2; i < arguments.length; i++) {  
      console.log("Zusätzlich:", arguments[i]);  
    }  
  }  
}
```

Aufruf mit vier Argumenten:

```
logData("John", "Doe", 24, "Doeville");
```

Argumente: 4
Vorname: John
Nachname: Doe
Zusätzlich: 24
Zusätzlich: Doeville

Aufrufen: Rest-Parameter



Definition einer Funktion mit Rest-Parameter:

```
function addValues(...numbers) {  
  let sum = 0;  
  for (let number of numbers) {  
    sum += number;  
  }  
  return sum;  
}
```

Aufrufe mit beliebiger Anzahl an Argumenten:

```
console.log("3 Argumente:", addValues(1, 2, 3)); // 6  
console.log("5 Argumente:", addValues(1, 2, 3, 7, 11)); // 24  
console.log("10 Argumente:", addValues(1, 2, 3, 7, 11, 81, 19, 2, 33, 10)); // 169
```

Standardparameter



Reguläre Definition:

```
function quadrat(x) {  
    return x * x;  
}
```

Aufruf ohne Argument:

```
quadrat(); // NaN
```

Definition mit Standardparameter:

```
function quadrat(x = 0) {  
    return x * x;  
}
```

Aufruf ohne Argument:

```
quadrat(); // 0
```

Gültigkeit von Variablen



```
let scope = "global";  
function f() {  
  let scope = "local";  
  console.log(scope);  
}
```

f(); ← local

console.log(scope); ← global

Gültigkeit von Variablen



```
let scope = "global";  
function f() {  
  scope = "local";  
  console.log(scope);  
}
```

f(); ← local

console.log(scope); ← local

Function vs. Block Scope



```
var scope = "global";  
function f() {  
    var enterBlock = true;  
    console.log(scope); // Was steht hier?  
    if (enterBlock) {  
        var scope = "local";  
    }  
    console.log(scope); // Existiert scope noch?  
}  
f();
```

undefined

scope ist in der gesamten Funktion gültig (hoisted variable)

Function vs. Block Scope



```
let scope = "global";  
function f() {  
  let enterBlock = true;  
  console.log(scope);  
  if (enterBlock) {  
    let scope = "local";  
    console.log(scope);  
  }  
  console.log(scope);  
}  
f();
```

global

local

global

Anyonyme & Arrow Funkt.



Anonyme Funktion (Zuweisung an Variable):

```
const calcAsAnonFunction = function(x) {  
  return x * x;  
};  
console.log("Anonyme Funktion:", calcAsAnonFunction(4));
```

Arrow Funktion:

```
const calcAsArrowFunction = (x) => {  
  return x * x;  
};  
console.log("Arrow Funktion:", calcAsArrowFunction(4));
```

Kurze Arrow Funktion (1 Parameter, Körper besteht aus 1 Statement):

```
const calcAsShortArrowFunction = x => x * x;  
console.log("Kurze Arrow-Funktion:", calcAsShortArrowFunction(4));
```



Global Functions

```
isFinite(number);
```

```
isNaN(value);
```

```
parseFloat(string);
```

```
parseInt(string, radix);
```

```
encodeURIComponent(uri);
```



Nächstes Mal in WEF1VO

JavaScript im Browser

Credits

- No credits to show today.

